LAPLACE: Predictive Sandbox Orchestration for LLM Agents

Yihui Zhang[1], Tianyu Wo[1], Jinghao Wang[1], Xiaoyang Sun[2†], Menghao Zhang[1], Cangzhou Yuan[1], Li Li[1], Chunming Hu[1], Albert Y. Zomaya[3], Renyu Yang[1†]
[1]Beihang University [2]University of Leeds [3]The University of Sydney

ABSTRACT

Modern Large Language Model (LLM) agents increasingly use the Model Context Protocol (MCP) to interact with external sandboxes. Serving these decoupled services efficiently faces a key tradeoff: reserving all sandboxes is too resource-intensive, while on-demand startup introduces cold starts that worsen tail latency in multi-tenant settings. We present LAPLACE, a predictive serving system that orchestrates tasks of an agent workflow efficiently. LAPLACE uses intent-aware prewarming that runs keyword matching and streaming semantic classification in parallel, detecting sandbox execution intent early and overlapping LLM inference with sandbox setup. To coordinate heavyweight sandboxes across workflow steps, LAPLACE performs context-aware stochastic prefetching over a sandbox dependency graph, predicting upcoming sandbox transitions to gain prewarm lead time. It further employs a semantic cache to reuse compatible results and avoid redundant execution, and an out-of-band data path with a shared-memory backplane to bypass the network stack for zero-copy data transfer. Under highly concurrent, multi-turn agent workloads, LAPLACE reduces P99 end-to-end latency by up to 2.9× versus the *on-demand* runtime, and uses 45.9% less peak memory than the *reserved* runtime.

CCS CONCEPTS

• **Computer systems organization → Cloud computing**; • **Software and its engineering → Distributed systems organizing principles**.

KEYWORDS

LLM Agent, Execution Runtime, Predictive Prewarm

ACM Reference Format:
Yihui Zhang, Tianyu Wo, Jinghao Wang, Xiaoyang Sun, Menghao Zhang, Cangzhou Yuan, Li Li, Chunming Hu, Albert Y. Zomaya, Renyu Yang. 2025. LAPLACE: Predictive Sandbox Orchestration for LLM Agents. In *ACM Symposium on Cloud Computing (SoCC '25), November 19–21, 2025, Online, USA.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3772052.3772264

Corresponding Author: Xiaoyang Sun (x.sun4@leeds.ac.uk), Renyu Yang (renyuyang@buaa.edu.cn).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SoCC '25, November 19–21, 2025, Online, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2276-9/25/11…$15.00
https://doi.org/10.1145/3772052.3772264

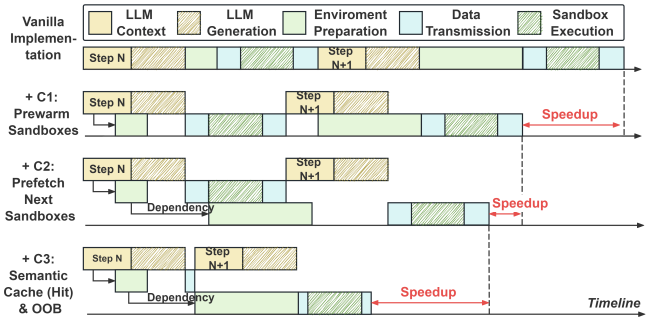


Figure 1: Proactive and overlapped execution v.s. the native vanilla approaches

1 INTRODUCTION

Modern LLM agents are evolving from conventional text generators into autonomous systems that iteratively reason, plan, and execute external actions [18, 39, 45]. Unlike traditional LLM serving workloads that primarily optimize token generation [2, 47], agent execution forms a stateful run-loop where an LLM-based controller repeatedly invokes external tools [7, 10, 26, 27] such as code execution, web automation, and data processing services. For security, isolation, and reproducibility, these tools are increasingly deployed as independent sandbox environments and accessed through standardized interfaces such as the Model Context Protocol (MCP) [5]. Consequently, the latency of agent execution is no longer determined solely by model inference, but also by the efficiency of coordinating heterogeneous external execution environments.

Modern cloud-native infrastructures increasingly adopt serverless sandbox execution to support large-scale concurrent agent sessions [4, 11, 25], i.e., the execution environments are initialized on demand rather than being reserved as long-lived instances. This design enables the required elasticity for multi-tenant agent workloads but introduces non-negligible latency [29, 35, 44, 46]. Each tool invocation may incur sandbox initialization overhead — such as image loading, filesystem preparation, namespace configuration, and runtime handshake — introducing a second-level delay before execution begins [12]. In multi-turn agent workflows, such startup overheads accumulate across successive tool invocations, significantly degrading end-to-end responsiveness and limiting the practicality of interactive agent services.

The main cause of this latency is not sandbox initialization, but the sequential execution model used by the vanilla implementation of the de facto agent runtimes (e.g., AgentScope[9], AutoGen[43], and LangGraph [15]). These systems use a reactive and sequential model without pipelined orchestration: the stage of sandbox preparation begins only after the stage of LLM inference finishes the token generation, and the tool invocation is fully determined. As a result, the preparation of the environment remains entirely exposed on the critical path, leaving idle periods between model reasoning and tool execution. While GPUs are occupied generating

tokens, CPU-side resources remain underutilized — once sandbox initialization begins, the GPU has to wait for external execution to complete. As illustrated in Fig. 1, the reactive model serializes LLM reasoning and environment preparation, without effectively overlapping computation with environment provisioning.

Existing runtime optimizations such as speculative execution [17, 36], prewarming [23, 37], workflow orchestration [20, 21], and caching [22, 44] are well studied in serverless and cloud systems. However, they do not directly transfer to LLM agent runtimes because agent execution differs fundamentally from conventional request processing. Prior work usually assumes fixed execution targets or pre-defined workflow graph before runtime. In contrast, autonomous agent workflows are produced online via auto-regressive reasoning: tool calls emerge progressively from streaming token generation and remain uncertain until enough semantic context is available. Multi-turn sessions further evolve based on intermediate observations and outcomes, making future tool choices both workflow-dependent and highly dynamic. These characteristics make simple extensions of existing techniques infeasible.

In the context of LLM agent runtimes, the key idea is therefore to pre-launch the most likely sandbox before it is requested, overlapping environment preparation with ongoing LLM generation. However, realizing this idea for autonomous agent systems introduces three unique challenges. First, within a single step (agent iteration), intents must be inferred from early token streams with only partial semantics; using short contexts with low thresholds or broad candidate sets improves overlap but risks false-positive prewarming and wasted memory/CPU. Second, across steps, predicting which tool will be invoked in future steps becomes increasingly unreliable as the prediction window extends; prewarming all plausible successors would revert to reserved-deployment costs. Third, prewarming alone removes only sandbox startup delay: repeated tool executions and large artifact transfers can still remain on the critical path even when a sandbox is ready. The system must therefore reuse semantically equivalent execution results and decouple bulk data transfer from control signaling, while preserving protocol compatibility and user-visible semantics.

This paper presents Laplace[1], a predictive serving system that orchestrates tasks of an agent workflow efficiently. Laplace overlaps LLM execution with environment preparation, breaking the rigid sequential dependencies in vanilla implementation of agent runtimes. Laplace diminish end-to-end latency through three synergetic techniques: i) *intent-aware sandbox prewarming* that speculates execution intents on the basis of streaming token outputs, at a proper time, and overlap sandbox preparation with ongoing generation within a step; ii) *stochastic sandbox prefetching*, leveraging historical agent execution traces to anticipate future sandbox needs and prepares the sandboxes for the next step during the current step, thereby lowering cold-start latency; and iii) *reuse-aware data transmission* that exploits semantic similarity to bypass redundant tool execution through semantic caching and decouples large artifact delivery from control signaling through an out-of-band data path, eliminating unnecessary computation and serialization overhead.

[1]Named after *Laplace Transform* and *Laplace's demon*, metaphorizing coordinated execution and predictive orchestration.

Laplace's prototype is implemented and integrated with the open-source AgentScope framework [9]. Laplace is framework-agnostic and highly extensible to any LLM agent serving infrastructures. On diverse multi-turn, high-concurrency agent benchmarks, Laplace outperforms state-of-the-art serverless runtimes while preserving workflow correctness and protocol compatibility. Laplace reduces P99 latency by up to 2.9× and peak host memory usage by 45.9%. This paper makes the following key contributions.

- characterizing the latency-critical path of multi-turn LLM agent execution and proposing a new sandbox prewarming approach (**C1**) via inferring streaming token-level intents such that sandbox preparation can be better overlapped with the ongoing LLM inference (§ 3.1).
- devising stochastic sandbox prefetching mechanism (**C2**) to reduce cross-step cold-start latency by mining historical agent execution traces and pre-warming likely sandbox environments over a sandbox dependency graph (§ 3.2).
- introducing reuse-aware data transmission (**C3**), via out-of-band transport and semantic caching, efficiently transferring and reusing intermediate execution artifacts while eliminating redundant sandbox initialization (§ 3.3).

2 BACKGROUND AND MOTIVATION

2.1 Agent Workflow and Environment

Large Language Model (LLM) applications have shifted from monolithic, single-turn chatbots to distributed autonomous agents. Traditional workflows rely on manually predefined execution graphs, where the operation sequence is fixed before runtime. In contrast, modern *agent workflows* integrate autonomous decision-making: given a high-level objective, an agent can dynamically decompose tasks, select execution sandboxes or external environments, and adapt its trajectory based on intermediate observations. As a result, the workflow is no longer statically specified but instead emerges from continuous interactions between LLM reasoning and external environments [18, 39, 45].

An agent workflow is realized through a sequence of event-driven execution steps, where each step represents an intermediate task that involves reasoning, planning, or invoking local and remote sandboxes. Tasks that require file system interaction, code execution, web access, or queries against proprietary databases are delegated to isolated external execution layers (*environment* or *tool sandboxes*). Standardized protocols—most notably the Model Context Protocol (MCP) [5] proposed by Anthropic—formalize this interface boundary. MCP specifies a common communication layer over transports such as HTTP and Server-Sent Events (SSE), thereby transforming tool invocation into a set of loosely coupled microservices. This architecture is closely aligned with emerging agent-oriented operating systems that conceptualize LLMs as central processing units and external environments as peripheral devices [9, 24]. From a systems-engineering standpoint, an agent's execution increasingly manifests as a continuous stream of heterogeneous, RPC-like interactions between the LLM inference core and multiple environment services, rather than as a monolithic, localized compute workload.

2.2 Serving Runtimes

Disaggregating Agent Engines from their underlying execution environments enables substantial scalability and architectural flexibility. However, deploying these decoupled components within cloud-native, multi-tenant cluster infrastructures introduces significant challenges related to performance and resource management. Service providers must carefully balance isolation guarantees, resource efficiency, and end-to-end latency, thereby exposing an inherent trade-off between long-running (reserved) and serverless (on-demand) serving paradigms.

Reserved Agent Runtime. A straightforward strategy [12, 38] is to maintain pre-initialized, always-on containers for each user tenant and its associated sandboxes. However, in contemporary agent ecosystems comprising thousands of fine-grained tools, sustaining all execution environments in a warmed state becomes prohibitively resource-intensive: idle containers incur substantial host memory and CPU overheads [35, 44], which in turn lead to pronounced cluster underutilization and resource interference, thereby rendering large-scale multi-tenant deployments economically unsustainable.

On-demand Agent Runtime. To enhance resource utilization, modern cloud-native platforms commonly employ on-demand runtime provisioning [8, 32], wherein tool sandboxes are instantiated ("cold-started") only upon explicit requests from agents. However, this design choice introduces substantial tail latency: the on-demand initialization of an isolated container incurs a series of serialized overheads, including container image download and extraction, network namespace setup, virtual file system mounting, and application-level handshake procedures. Under bursty workloads or in complex multi-turn workflows, these multi-second cold-start delays accumulate along the agent's end-to-end critical path, thereby inflating P99 tail latency and degrading the interactive quality of service (QoS) of real-time intelligent agents.

2.3 Execution Bottlenecks in Agent Runtimes

In contrast to conventional LLM serving, an agent session constitutes a continuous, stateful execution loop composed of multiple iterative reasoning and sandboxed execution steps. Prevailing agent frameworks [9, 15, 43] employ a reactive execution paradigm, in which each tool invocation is initiated only after the LLM has completed its reasoning step. This design induces a strictly serialized dependency chain between model inference and environment interaction. The resulting execution workflow is depicted in Fig. 1.

As illustrated in Fig. 2a, the end-to-end latency of a single execution step N can be broken down into several parts:

$$T_{step}^{(N)} = T_{context}^{(N)} + T_{generation}^{(N)} + T_{env_prep}^{(N)} + T_{data_io}^{(N)} + T_{sandbox_exec}^{(N)} \quad (1)$$

The first two components are associated with LLM inference, including context processing and token generation. The remaining three define the optimization boundary for an agent runtime: $T_{env_prep}^{(N)}$ is the time duration to get the selected sandbox environment ready, $T_{data_io}^{(N)}$ is the time to exchange invocation inputs and results, and $T_{sandbox_exec}^{(N)}$ is the time spent executing the tool.

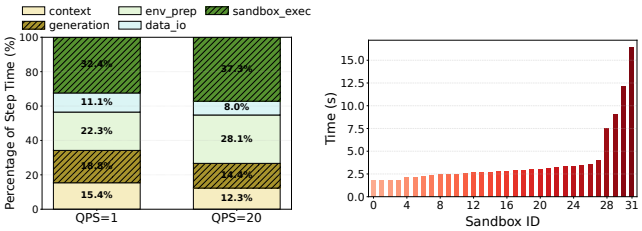
Laplace targets these time frames without changing the LLM inference stack by overlapping preparation with reasoning, reducing data movement, and eliminating redundant executions. We derived the following observations that motivate this study.

Observation #1: *In-step preparation can overlap with token generation.* In today's reactive execution model, environment initialization cannot start until the LLM emits a explicit sandbox invocation, even though the prompt, plan, and partial token stream often already reveal the likely sandbox. Using this partial evidence to start preparation earlier can put the prewarm even forward, ahead of the corresponding invocation. This interval lets sandbox startup, runtime connection setup, and other preparation work overlap with the remaining $T_{generation}^{(N)}$, rather than placing all of $T_{env_prep}^{(N)}$ on the critical path. As shown in Fig. 2b, sandbox initialization requires several seconds and can approach 20 seconds for heavyweight sandboxes, rendering this overlap mechanism critical for minimizing the effective preparation latency observed by the system.

Observation #2: *Cross-step execution can advance the prewarming.* Consecutive steps within an agent session frequently exhibit strong temporal locality. For instance, in a large language model (LLM) serving workload, a `paper search` step is often followed by a `document reading` step that processes a retrieved document, and a `data analysis` step is commonly followed by a `figure generation` step. Nonetheless, prevailing runtime systems generally treat each invocation as an independent event, discarding workflow context once a step is initiated. In contrast, the runtime can exploit the execution window of the current sandbox to proactively instantiate and prewarm the most likely subsequent execution environments before step $N+1$ begins, thereby making the intra-step prewarm more in-advance beyond the current decoding stage.

Observation #3: *Redundant execution and excessive unnecessary data transfer.* In multi-turn agent workflows, later reasoning steps often revisit information that was already produced, such as querying the same document again or re-running deterministic analyses. However, conventional runtimes still re-execute these operations even when an equivalent result is already available, repeatedly paying the cost of $T_{sandbox_exec}^{(N)}$. Meanwhile, sandbox interfaces often couple the synchronous transfer of large intermediate artifacts (e.g., files, images, structured outputs) with control messages, causing $T_{data_io}^{(N)}$ to scale with artifact size. Intuitively, one can accelerate the agent execution by caching and reusing prior execution results and decoupling artifact transmission from control signaling, thereby eliminating redundant work and data transfer overhead.

These observations reveal optimization opportunities at three points in the agent execution loop: overlapping the environment preparation with the LLM operations within a step, advancing the environment preparation of the next step in the current step, and eliminating repeated execution or data movement. Existing LLM serving frameworks primarily focus on optimizing model execution — such as prefill and decoding phases, as well as KV cache management — while serverless runtimes primarily reduce the overhead associated with container initialization without considering user-wise intent. However, neither of them directly addresses how an agent-oriented runtime can leverage these optimizations while maintaining resource efficiency and preserving the intended semantics of tool invocation and usage.



(a) Step time breakdown. (b) Sandbox cold-start latency.

Figure 2: Execution bottlenecks in agent runtimes. (a) Execution-step latency breakdown. (b) Mean cold-start latency across 32 sandboxed environments, including all MCPBench [40] sandboxes and additional commonly used environments. Most sandboxes initialize in 2–4 seconds, while resource-intensive environments can require up to approximately 20 seconds.

2.4 Research Challenges

Designing an effective execution runtime to accelerate the LLM agent serving is faced with the following challenges.

Challenge #1: *Early intent prediction vs. resource waste.* Intent speculation of tool use from given incomplete token stream is critical to ensure the timeliness of sandbox environment preparation whilst avoiding unwanted tool launch due to inaccurate intent prediction. Early warmup must rely on highly ambiguous early-stage tokens (e.g., general-purpose verbs such as `read` and `get` reused across different MCP tools), which often causes *false-positive* sandbox activation, wasting resources and potentially triggering host-level OOM in multi-tenant deployments.

Challenge #2: *Cross-step in-advance prewarming vs. non-deterministic workflow execution.* It is advantageous to select the subsequent-step sandbox environments in advance of the manifestation of the next intent. The agent workflow executes in a probabilistic manner, admitting multiple plausible successor states. Prewarming all potential successors incurs a computational and resource overhead comparable to that of a fully reserved deployment, whereas prewarming only a single successor substantially reduces the opportunity to amortize sandbox initialization costs and yields only limited latency improvements. Instead, the runtime system should exploit historical transition data to rank a small set of highly probable target environments, then refine this choice using the signal available at the current step. This balances resource utilization against latency reduction while preserving the ability to conceal sandbox initialization behind ongoing computation.

Challenge #3: *Reuse and data-path efficiency vs. sandbox compatibility.* We must skip redundant work without changing what the agent observes from a sandbox invocation. Exact reuse is safe but misses semantically equivalent requests expressed differently; unconstrained semantic reuse may yield incompatible results. Similarly, putting large artifacts in normal RPC messages preserves compatibility but keeps data movement proportional to payload size, while replacing the control interface would break existing tools. Hence, the runtime must detect compatible reuse, reject unsafe approximations, and decouple bulky artifacts from control signaling while preserving sandbox-execution semantics.

3 DESIGN OF LAPLACE

We focus on three critical time periods within the execution of reactive agents: environment preparation, data movement, and sandbox execution. This section how Laplace accelerate them with decoupled yet inter-connected optimizations: intent-aware sandbox prewarming that infers an in-step tool intent early to overlap the environment preparation with the decoding of LLM serving (§ 3.1); stochastic sandbox prefetching that exploits cross-step workflow regularity to further put the preparation forward (§ 3.2); and reuse-aware data transmission that avoids redundant execution and removes large artifacts from the control path (§ 3.3).

3.1 Intent-Aware Sandbox Prewarming

Naive reactive runtimes await a full tool invocation before sandbox preparation, even if prompts, evolving plan, and partial generation already reveal user intents. Laplace instead prewarms the sandbox earlier by inferring streaming token-level intents, better overlapping sandbox preparation with ongoing LLM inference.

Laplace makes the best use of the LLM docoding period $T^{(N)}_{generation}$ to obtain *just enough* user intents for launching accurate sandboxes without compromising the timeliness of prewarming. This is done by streaming context to two independent routers. As shown in Fig. 4, there exists a dilemma: earlier intent predictions overlap more with the LLM decoding period but are less reliable and can waste sandbox capacity. Keyword Router acts early but must balance false positives against a stricter threshold; Semantic Router is more precise but slower. Requiring both predictions incurs the Semantic Router's delay, whereas opportunistically combining their outputs preserves high precision while allowing earlier decisions. Hence, Laplace proposes an intent-aware sandbox prewarming mechanism that combines the two asynchronous intent predictions, rather than considering either one on its own to be sufficient.

Keyword Router. The Keyword Router scans the stream against tool-specific keyword profiles and can emit a candidate within microseconds of a distinctive token, preparing the environment while the LLM continues decoding. Common terms like `research`, `search`, or `slide` occur in many tool descriptions: triggering on a single match creates a large prewarm set with many false positives, while requiring many matches will delay the preparation, missing good chances of overlapping. We therefore apply a threshold γ on the number of matched tool-specific keywords. In Fig. 3, surface cues in the user request and evolving plan let the Keyword Router emit likely sandbox candidates before the plan agent finishes the current ReAct step. The threshold balances early activation with resource waste; the chosen configuration is given in § 5.3.

Semantic Router. In parallel, the Semantic Router compares the active context with tool-intent representations. It captures requests whose wording overlaps little with a tool profile and disambiguates generic keyword cues using the plan and prior generation. In Fig. 3, it can identify candidate sandboxes from the broader task intent even when no single token uniquely identifies a tool. Its tradeoff is temporal: reliable semantic evidence typically needs a longer prefix, so a semantic-only decision often arrives too late to hide the cold start latency of a heavy-weight sandbox such as PaperSearch [28], and Neo4j [27]. The Semantic Router is an asynchronous, complementary source of candidates that can recover intents the Keyword Router misses.

Union Assembly. Let $\mathcal{S}^{(N)}_{key}$ and $\mathcal{S}^{(N)}_{semantic}$ be the candidate sets from the two routers at token step N. The lower-left quadrant of

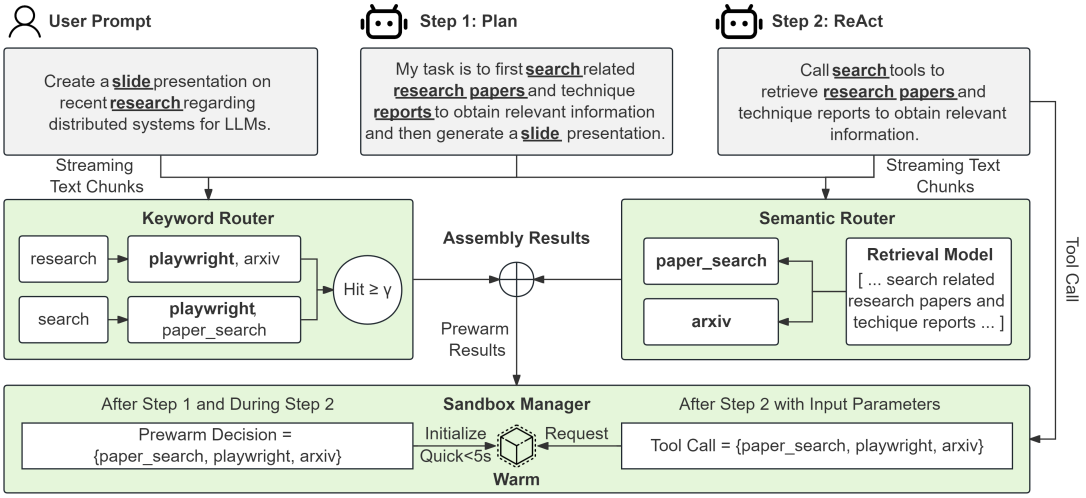


Figure 3: Intent-aware sandbox prewarming: the plan agent expands user requests into ReAct steps, and keyword plus semantic predictions jointly decide to prewarm the sandbox or not.

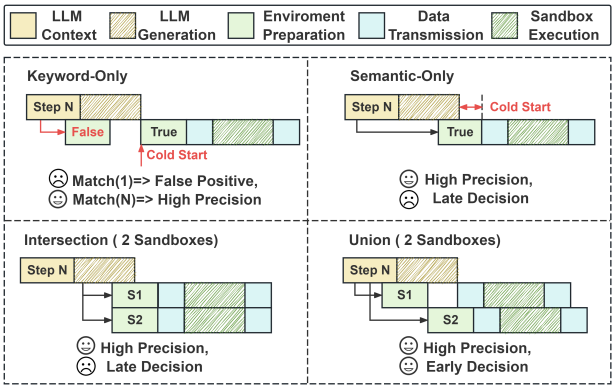


Figure 4: Tradeoffs among routing policies. Keyword routing trades early decisions for potential false positives; semantic routing and intersection delay a precise decision; union assembly retains precision while allowing either router to trigger prewarming early.

Fig. 4 shows why prewarming only their intersection is unsuitable: it boosts apparent precision but makes every trigger wait for the Semantic Router and drops valid tools whenever either router has imperfect recall. Instead, we use the lower-right policy:

$$\mathcal{S}^{(N)}_{trigger} = \mathcal{S}^{(N)}_{key} \cup \mathcal{S}^{(N)}_{semantic}. \tag{2}$$

Each router manages its own false positives, and the union lets the first credible signal start preparation. Fig. 3 shows the resulting behavior: routers independently generate candidates from the request and partial plan, then form a unified prewarm set for the sandbox manager. Explicit intents benefit from the early prewarm of the Keyword Router, while implicit intents are handled by the Semantic Router. As will be shown in § 5.3, asynchronous union can minimize waiting time and eliminate cold starts in the evaluated routing workload.

3.2 Stochastic Sandbox Prefetching

Intent-aware sandbox prewarming can only use the remaining decoding time of the current step, which is inadequate when a non-resident sandbox takes longer to start than the marginal token-generation window. In reality, steps in an agent workflow are non-deterministic but generally follow a probabilistic model. For instance, after a paper search, an agent may read a returned document; after data analysis, it may generate a figure or report. LAPLACE exploits these cross-step probabilistic patterns to navigate the environment preparation in advance during $T^{(N)}_{sandbox_exec}$, before the next step commits to a specific invocation.

Stochastic Markov Process Modeling. Let $\mathcal{V}$ denote the sandbox state space, where each state is a sandbox type or a typed tool-state tuple in the state-level SDG. For an execution trace $\{S^{(1)}, \ldots, S^{(N)}\}$, LAPLACE records step-to-step transitions in a directed sandbox dependency graph (SDG). For each ordered pair (v_i, v_j), we maintain transition counts:

$$C_{i,j} \leftarrow C_{i,j} + \mathbf{1}\left[S^{(n)} = v_i,\ S^{(n+1)} = v_j\right] \tag{3}$$

The next-state probability is estimated by a first-order Markov model with *Laplace Smoothing* [16]:

$$P_{i,j} = P\left(S^{(n+1)} = v_j \mid S^{(n)} = v_i\right) = \frac{C_{i,j} + \alpha}{\sum_{k \in \mathcal{V}} (C_{i,k} + \alpha)}, \tag{4}$$

where $\alpha = 1$ avoids zero-probability collapse for sparsely observed transitions.

Prefetching Under Budget Constraints. Given current state v_i, LAPLACE first identifies sandbox candidates with non-trivial cold-start cost:

$$\mathcal{K}_i = \left\{v_j \in \mathcal{V} \setminus \{v_i\} \,\middle|\, L_j \geq \lambda\right\}, \tag{5}$$

where L_j denotes the estimated cold-start penalty of sandbox v_j and λ is a lightweight cost threshold.

LAPLACE then filters out low-confidence successors and retains only high-probability candidates:

$$C_i = \left\{v_j \in \mathcal{K}_i \,\middle|\, P_{i,j} \geq \tau\right\}, \tag{6}$$

where τ controls false-positive prewarming. Finally, LAPLACE selects the top-B sandboxes under a fixed budget:

$$\mathcal{A}_i = \text{top-B}\left(C_i,\ P_{i,j}\right), \tag{7}$$

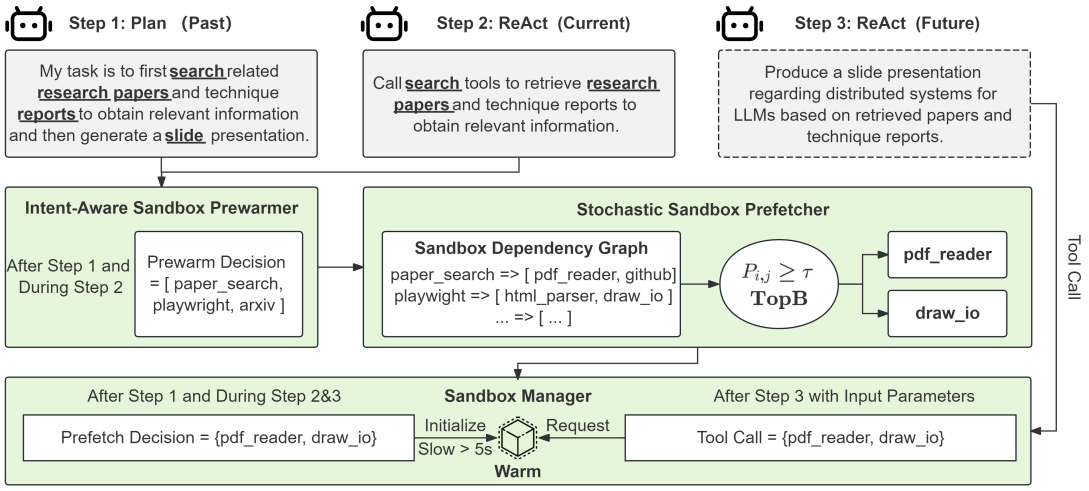


Figure 5: Stochastic sandbox prefetching in Laplace. Execution history predicts likely sandboxes for subsequent ReAct steps.

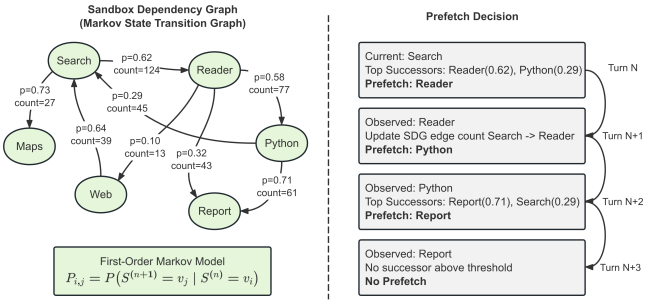


Figure 6: Stochastic sandbox prefetching using a first-order Markov model. Left: SDG-based Markov state transition graph with edge probabilities from observed counts. Right: example session showing thresholded, budgeted prefetching across turns, where predicted successors are prewarmed before the next step commits.

where C_i is the filtered candidate set, $\mathcal{A}_i$ is the final budgeted prefetch set, and B bounds the number of sandboxes prewarmed per step. This policy is intentionally lightweight and can be executed outside the LLM generation critical path.

Online Update. After each completed sandbox invocation, Laplace appends one transition edge to SDG and updates $C_{i,j}$ and $P_{i,j}$ asynchronously in the background prefetch worker. Thus, the predictor continuously adapts to evolving multi-turn workflows without blocking foreground agent execution. Fig. 5 shows how stochastic sandbox prefetching operates together with intent-aware sandbox prewarming in a complete multi-step workflow. After step N finishes in sandbox v_i, Laplace queries the SDG for likely successor sandboxes, filters out low-value or low-confidence candidates using $L_j \geq \lambda$ and $P_{i,j} \geq \tau$, and then selects a budgeted prefetch set $\mathcal{A}_i$ before step $N+1$ is fully committed.

3.3 Reuse-Aware Data Transmission

Once an invocation is ready, its remaining critical-path cost lies in artifact movement ($T^{(N)}_{data_io}$) and tool execution ($T^{(N)}_{sandbox_exec}$). Laplace first checks if it can reuse a cached result; otherwise, it runs the tool and returns its artifact via a separate data path. Fig. 7 details this procedure.

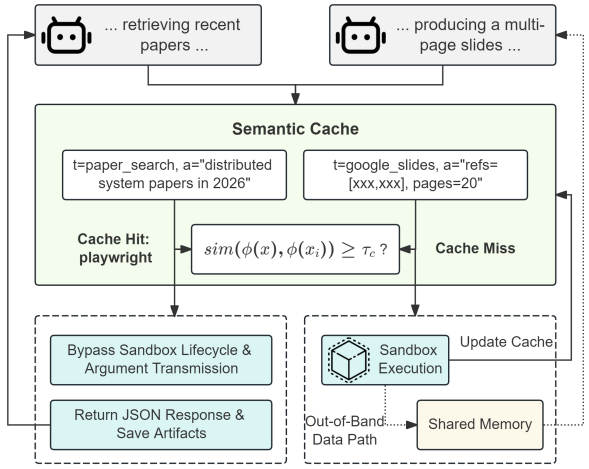


Figure 7: Reuse-aware data transmission in Laplace. A semantic-cache hit returns a prior result (paper_search); a miss executes the sandbox (paper_slides) and delivers the result through the out-of-band data path.

Semantic Caching. Multi-turn agents often re-access unchanged documents, submit semantically equivalent queries with different wording, or re-run deterministic computations. Exact argument matching is robust but fails to capture such superficial variations, while unconstrained semantic matching can mistakenly merge requests that differ in tools, inputs, or side effects. Thus, caching must broaden the set of reusable requests while strictly preserving functional equivalence and behavioral compatibility.

For each completed deterministic invocation, Laplace stores a normalized invocation signature and its result. During lookup, it first filters cache entries by tool identity, then compares normalized invocation representations are compared against cached signatures. For a request x, result reuse is allowed only if a compatible cache entry exists that satisfies the semantic equivalence constraint:

$$\text{hit}(x) = \exists i \text{ s.t. } tool(x) = tool(x_i) \ \wedge \ sim(\phi(x), \phi(x_i)) \geq \tau_c, \quad (8)$$

where x denotes the incoming tool invocation request and x_i denotes the i-th cached invocation. Function $\phi(\cdot)$ transforms an invocation into a normalized semantic representation by first removing superficial variations in argument representation and then extracting semantic features for similarity comparison. Function $sim(\cdot,\cdot)$ measures the similarity between two invocation representations, and the threshold τ_c controls the strictness of semantic reuse. The tool identity constraint guarantees interface-level compatibility, while the similarity threshold limits reuse to invocations with sufficiently similar normalized semantics.

This design treats semantic matching as a conservative extension of exact reuse rather than an unconstrained approximation mechanism. Semantic similarity only enlarges the reusable request space within the boundary of the same tool interface and deterministic invocation behavior. On a cache miss, execution proceeds along the standard path and appends the fully computed result to the cache. On a cache hit, both sandbox initialization and tool invocation are skipped, thereby reducing $T^{(N)}_{env_prep}$ and $T^{(N)}_{sandbox_exec}$. As will be demonstrated in § 5.3.3, the semantic caching strategy can recover a larger proportion of redundant computation than using the strategy of exact matching alone, while maintaining a conservative fallback path that preserves the inference correctness.

Out-of-Band Data Transmission. Even after a sandbox is fully initialized, conventional RPC transports serialize large logs, files, images, and structured outputs into request–response messages. Consequently, $T^{(N)}_{data_io}$ scales with the payload size and blocks the agent prior to its subsequent reasoning step. A wholesale replacement of the control protocol would compromise compatibility with existing MCP tools; therefore, Laplace instead decouples control signaling from artifact transfer.

The in-band control plane continues to carry standard tool directives, completion notifications, error reports, and compact metadata. Large artifacts are represented on this plane solely by a fixed-size reference and are exchanged via a co-located, zero-copy data path. Under this design, a cache miss executes as usual, publishes its artifact exactly once, and returns a reference to the Agent Engine; a cache hit simply reuses and returns the previously published artifact over the same data path. This architectural decoupling maintains the existing control interface while eliminating RPC-layer serialization and memory copying for high-volume outputs. As demonstrated in § 5.3.4, the corresponding transmission latency becomes effectively insensitive to payload size.

4 IMPLEMENTATION

Laplace is implemented as a predictive serving runtime between the agent engine and external execution sandboxes. Figure 8 shows its control and data paths. The implementation observes an agent run-loop without changing its tool-facing interface, and overlaps runtime work with the LLM and sandbox work already in progress.

Control Plane. On the control plane, a controller subscribes to the incoming token stream and records completed tool transitions. The Keyword Router matches the stream against tool keyword profiles and emits a candidate when the number of matched tool-specific keywords reaches $\gamma = 2$, the operating point selected in Section 5.3.

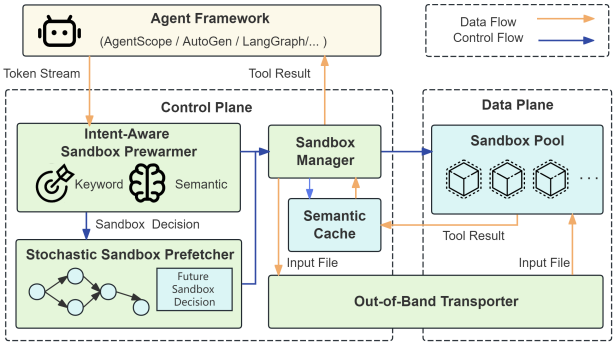


Figure 8: Overview of Laplace.

In parallel, the Semantic Router uses the sparse retrieval configuration selected in that section. Union Assembly dispatches their candidate sets independently to the sandbox manager, which deduplicates requests and starts the corresponding containers. A background prefetch worker updates the sandbox dependency graph from execution traces, applies the transition probabilities in Section 3.2, and submits budgeted next-step warmups while the current tool is executing. In our implementation, we configure the cost threshold as $\lambda = 5$. prefetch probability threshold as $\tau = 0.6$ and the per-step prefetch budget as $B = 1$.

Data Path. Before scheduling a deterministic invocation, the runtime constructs its invocation representation, restricts lookup to the same tool identity, and checks its semantic-result cache. Each cache entry contains the tool identifier, normalized invocation signature, semantic embedding, and result reference. The cache reuses a previous result only when the tool identity matches and the semantic similarity exceeds the threshold $\tau_c = 0.8$ used in the reported experiments. On a miss, the sandbox writes a large result into a host-managed memory-mapped shared-memory region. The control plane carries only a 64-bit `token_id` that names this region, while the Agent Engine reads the result directly from the shared-memory backplane. Small directives, metadata, completion notifications, and errors remain on the normal control plane. This implements the common cache-or-execute flow in Figure 7 without serializing bulky results through the RPC stack.

Execution boundary and correctness. We consider multi-tenant agents whose tools run in OS-isolated containers or microVMs. Predictive preparation makes an environment ready but does not execute side-effecting work before the agent commits the invocation; unused warmups are discarded. Cache reuse is limited to deterministic, compatible tool requests. The shared-memory bridge operates inside one trusted host or securely managed cluster boundary, with existing access controls preventing cross-tenant memory access. Sandbox escapes, malicious agents, and compromised infrastructure are outside this work's threat model.

5 EVALUATION

5.1 Experiment Setup

Hardware and Software Environment. All experiments are conducted on a commodity server equipped with an 16-core CPU,

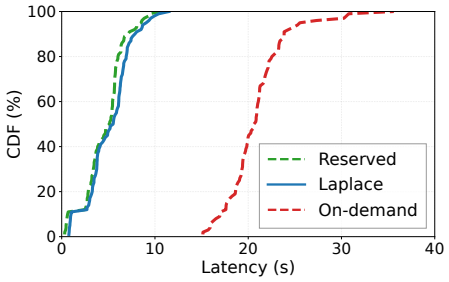


Figure 9: Cumulative sandbox provisioning latency across multi-turn agent sessions (5–8 steps per session).

256 GiB of host memory, and a 2 TB NVMe SSD. Laplace is built upon the AgentScope [9] framework, specifically integrated with its agent engine layer. The isolated multi-tenant execution sandboxes are instantiated via Docker containers, and the entire runtime infrastructure is implemented in Python. To drive agent reasoning, we utilize Alibaba DashScope's Qwen3.5-Max model [3], accessed concurrently via its production cloud API endpoints.

Workloads. We evaluate Laplace using a trace-level benchmark with 200 multi-turn trajectories, derived from MCPBench [40] with 32 open-source MCP-compatible tool servers (e.g., Playwright [26], Jupyter [7], Neo4j [27]) collected from GitHub [10]. To ensure tool-level validity and realistic workflow execution dependencies, we construct the benchmark in two stages:

- **Atomic Tool-use Generation:** We generate 20 single-turn interaction templates per tool (640 in total) to cover diverse atomic behaviors across all 32 tools.
- **Multi-turn Trajectory Construction:** An LLM planner incrementally generates 1–10 step agent sessions, where each planned task is executed against actual servers to get real execution results back into subsequent planning steps.

The resulting dataset is uniformly distributed with 20 traces per trajectory length, averaging 6.4 steps per session and 2.96 tools per step. This execution-grounded approach ensures tool-level validity and eliminates tool-specific bias while reflecting representative LLM agent workflows.

Methodology. Unless otherwise specified, we evaluate all systems by replaying full traces as session-level workloads with a fixed random seed (`seed = 0`). Each trace is executed in a step-wise manner, preserving dependencies across reasoning, tool invocation, and sandbox execution to faithfully model realistic multi-turn agent workflows. Under this deterministic sampling setting, the resulting trace distribution naturally exhibits a concentration in the 5–8 step range, which we therefore treat as the representative workload regime rather than a manually selected subset.

We evaluate all experiments at the session level unless explicitly stated otherwise, and defer workload-specific configurations (e.g., trace length or sampling range) to each individual ablation study.

Metrics. We evaluate Laplace across three dimensions aligned with system design goals:

- **End-to-end Latency:** Per agent session, we report mean and tail latency (P99) to capture both average performance and long-tail behavior under multi-step execution.
- **Resource Efficiency:** We measure peak CPU and memory consumption under multi-tenant workloads to quantify the runtime overhead introduced by sandbox provisioning, execution, and lifecycle management.
- **Prediction Accuracy:** We evaluate the accuracy of all predictive runtime optimizations, including intent-aware sandbox prewarming, stochastic sandbox prefetching, and semantic cache. For each, we report i) correct triggering or retrieval decisions under uncertain tool intent, and ii) false positive rate, reflecting unnecessary or incorrect activations such as misrouted intents, redundant prewarming.

5.2 End-to-End Performance

This section evaluates the end-to-end (E2E) performance of Laplace under two representative baselines in agent serving. We define our baselines as follows:

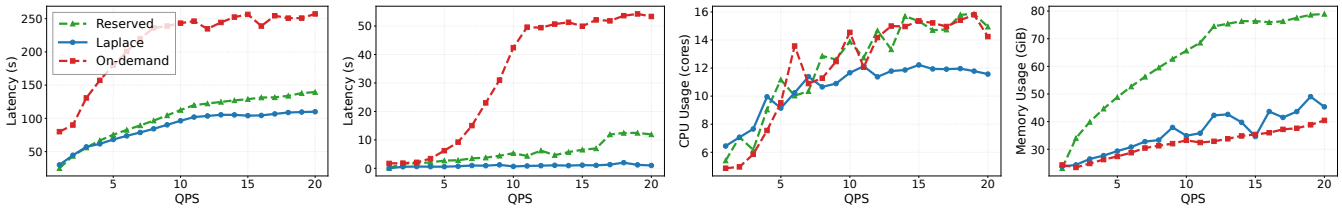
- **Reserved Runtime:** Maintains permanently warm sandboxes for all candidate tools. This represents the latency lower bound (performance ceiling). However, it is financially and physically non-viable in production due to prohibitive idle memory footprint across massive, multi-tenant MCP tool ecosystems.
- **On-demand Runtime:** Instantiates sandboxes dynamically upon tool calls, mirroring production MCP runtimes [4, 11, 25]. Since its bottleneck stems from application-level tool binding and session handshakes rather than generic OS booting, it remains the standard practical baseline.

We aim to answer a fundamental question: *Can Laplace successfully decouple execution latency from physical resource constraints, achieving serverful-like performance with serverless-like cost?*

5.2.1 Cumulative Sandbox Provisioning Latency in Multi-Turn Agent Workflows. We evaluate the end-to-end impact of Laplace on sandbox provisioning latency over multi-turn agent execution traces. We define cumulative sandbox provisioning latency as the sum of sandbox initialization delays incurred at each tool invocation within a session, focusing exclusively on environment setup overheads and excluding in-sandbox computation time.

Fig. 9 shows the distribution of cumulative latency across sessions. The *On-demand* baseline exhibits steadily increasing latency over longer execution horizons due to repeated cold-start overheads, resulting in a heavy-tailed distribution. In contrast, Laplace significantly reduces cumulative latency and maintains a much tighter distribution concentrated in the sub-second range. Compared to the *On-demand* baseline, Laplace achieves a 4.53× reduction in cumulative sandbox provisioning latency. Against the *Reserved* baseline, Laplace remains within a 10.6% performance gap while avoiding the substantial resource overhead of persistent sandbox allocation. These results demonstrate that predictive orchestration can effectively approximate *Reserved* execution performance under a *On-demand* deployment model.

5.2.2 Scalability. Fig. 10a summarizes the scalability of the three runtimes under different concurrency, with QPS increased from 1 to 20. Across low-to-mid concurrency levels, Laplace consistently



(a) P99 E2E Latency (b) Cumulative Provisioning Latency (c) Peak CPU Usage (d) Peak Memory Usage

Figure 10: End-to-end performance and resource consumption under concurrent workloads: (a) P99 E2E latency, (b) mean cumulative sandbox provisioning latency, (c) peak CPU usage, and (d) peak memory usage.

remains a latency profile close to *Reserved* while reducing end-to-end delay compared with *On-demand*. At higher concurrency, the advantage over *On-demand* remains substantial: at QPS=20, Laplace achieves 88.7s P99 E2E latency, a 2.9× speedup over *On-demand* (257.2s). This indicates that LAPLACE can absorb increasing concurrency without inheriting the long-tail latency explosion of the *On-demand* baseline.

Fig. 10b also reveals that the main scalability bottleneck of *On-demand* lies in cumulative sandbox provisioning. In the low-QPS regime, the mean cumulative sandbox provisioning latency stays below 4 seconds for all three modes. However, once the load reaches the higher-QPS regime (QPS $\geq$ 5), *On-demand* suffers growing degradation from network contention and resource constraints, and its cumulative sandbox provisioning latency rises steadily and eventually surpasses 50 seconds. By contrast, *Laplace* remains tightly bounded in the sub-5-second range and continues to slightly outperform *Reserved* in provisioning latency, reflecting the benefit of LAPLACE's intent-aware sandbox prewarming and stochastic sandbox prefetching under concurrency pressure.

5.2.3 Resource Footprint and Efficiency. Fig. 10c shows that the CPU footprint of the three runtimes remains relatively compact under low-load conditions but diverges significantly as concurrency scales. While *LAPLACE* exhibits no obvious advantages under low QPS loads, its strengths gradually emerge in the high-QPS regime where QPS $\geq$ 8. It consistently restrains peak CPU utilization across all tested loads, with the peak resource consumption capped at around 12.2 cores. This translates to a 22.8%–23.3% reduction in peak CPU pressure compared to both *On-demand* and *Reserved*, which demand roughly 15.8–15.9 cores. Particularly in the high-QPS regime, *LAPLACE* stabilizes within a tight band of $\sim$ 11–12 cores, whereas the two baselines fluctuate at a higher plateau of 14–16 cores, demonstrating that LAPLACE effectively mitigates CPU contention (saving up to 25% CPU resource under peak load) without sacrificing sub-millisecond control-plane responsiveness.

The memory footprint (Fig. 10d) exhibits a pronounced resource separation across the evaluation spectrum. As expected, *Reserved* is the most memory-intensive variant due to its persistence strategy; its memory consumption surges from an initial 24.6 GiB at light load to a massive peak of 80.6 GiB, staying sustained above 60 GiB as concurrency intensifies. Conversely, *On-demand* maintains a lean profile, bounding its peak usage between 24.3 GiB and 40.4 GiB. Notably, *LAPLACE* closely mirrors the trajectory of *On-demand* across the majority of QPS configurations, topping out at 49.4 GiB. This represents a 45.9% reduction in peak memory footprint compared to *Reserved*, demonstrating that LAPLACE successfully eliminates the prohibitive memory holding costs of long-run sandboxes, while maintaining a predictable, serverless-like resource elasticity under heavy concurrent workflows.

Table 1: Keyword threshold sensitivity ($\gamma = 1, 2, 3$).

γ	Avg Wait (ms)	Match Rate	Mismatch Runs
1	786.619	80.0%	20
2	323.021	95.0%	5
3	621.812	97.0%	3

5.3 Micro-Benchmarking

This section provides mechanism-level attribution for the end-to-end improvements reported above. In particular, we isolate the effects of intent-aware sandbox prewarming, stochastic sandbox prefetching, semantic caching, and out-of-band data transmission to avoid conflating component contributions in the E2E section.

5.3.1 Effectiveness of Intent-Aware Sandbox Prewarming. We further decompose intent-aware sandbox prewarming into three orthogonal factors: the keyword trigger threshold, the semantic model family, and the hybrid fusion policy. All results are measured on the same sampled tasks and repeated 100 times.

Sensitivity of Keyword Router Threshold γ. To rigorously isolate the impact of the keyword matching threshold, we evaluate $\gamma \in \{1, 2, 3\}$ under identical workload profiles, executing 100 independent trials for each configuration to ensure statistical convergence. As quantified in Table 1, $\gamma = 2$ emerges as the optimal configuration for the Keyword Router. Specifically, $\gamma = 2$ substantially compresses the average keyword-driven waiting latency to 323.0 ms (a 2.43× reduction compared to 786.6 ms under $\gamma = 1$) while maintaining a high predictive routing coverage of 95%.

The severe latency degradation under $\gamma = 1$ is primarily driven by its hyper-sensitivity, which triggers 20 distinct tool mismatch instances across the 100 runs; these false positives inadvertently amplify cold-start fallback penalties and introduce heavy tail latencies. Conversely, while increasing the threshold to $\gamma = 3$ further sharpens routing precision (yielding a 97% match rate with only 3 mismatch instances), it severely erodes the prewarm lead time, causing the average waiting time to climb back to 621.8 ms. This empirical trade-off confirms that $\gamma = 2$ effectively balances early predictive agility with resource stability.

Sensitivity of Semantic Router Models. Table 2 compares the performance and computational trade-offs of three semantic routing models under their optimal configurations:

Table 2: Semantic router model comparison (best operating point per model).

Model	Micro-F1	Hit Rate (Top-3)	Precision	Avg Latency (ms)
Retrieval	0.970	0.992	0.942	2.116
Encoder	0.776	0.968	0.634	7.822
FastText	0.124	0.124	0.124	0.171

- **Retrieval:** This model is implemented as a non-neural, sparse token-level retriever. It leverages TF-IDF [34] weighting combined with a *Top-N* nearest neighbor aggregation network, constructing composite vector features across token unigrams, selective bigrams, and character *n*-grams (ranging from 3 to 5 characters).
- **Encoder:** This neural model is built upon a fine-tuned *all-MiniLM-L6-v2* [31] transformer sequence representation network. It maps fluid multi-turn trajectories into dense vector spaces via contrastive pair-wise optimization under a *CosineSimilarityLoss* constraint.
- **FastText:** This lightweight model [13] represents text as the average of word and subword embeddings followed by a linear classifier.

Empirically, as quantified in Table 2, both *Retrieval* and *Encoder* models achieve a optimal prewarming Hit Rate, demonstrating that both sparse lexical tokens and dense semantic features can successfully capture all necessary tool invocation targets. However, *Retrieval* achieves higher orchestration quality and routing precision, yielding a Micro-F1 of 0.970 and a Precision of 0.942, whereas *Encoder* degrades to a Micro-F1 of 0.776 and a Precision of 0.634.

Furthermore, from an runtime efficiency perspective, *Retrieval* operates with a significantly lower computational footprint, executing with an average inference latency of only 2.116 ms–yielding a 3.70× speedup compared to the 7.822 ms latency incurred by *Encoder*. Conversely, while *FastText* provides ultra-low execution latency (0.171 ms), its shallow token representation space fails to deliver usable discrimination under multi-turn reasoning context drifts, resulting in a Micro-F1 and Hit Rate of merely 0.124. Consequently, *Retrieval* is selected as the production instance for Laplace's semantic branch, as it strikes the optimal Pareto-efficiency frontier between high-fidelity prediction accuracy and low-overhead control-plane latency.

Sensitivity of Assembly Policies. To rigorously isolate the algorithmic impact of the intent combination layer, we evaluate three distinct fusion policy paradigms under identical multi-turn context distributions:

- **Union (∪):** The predictive prewarming primitive is non-blocking and asynchronously dispatched the exact microsecond either the keyword router or the semantic router hits their respective individual activation boundaries.
- **Intersection (∩):** Predictive orchestration strictly enforces a dual-router consensus; a sandbox is prewarmed if and only if both the keyword router and semantic router concurrently validate the sandbox candidate's invocation intent.
- **Weighted:** This hybrid policy aggregates multi-modal intent metrics into a centralized candidate score: $s(c) = w_k \cdot s_k(c) + w_s \cdot s_s(c)$, where $s_k(c)$ denotes the matched-keyword ratio (normalized against the static lexicon scale) and $s_s(c)$ represents the real-time semantic retrieval similarity vector. A predictive trigger is dispatched only when clearing a rigid threshold: $s(c) \geq \tau_f$. In our empirical runner, we implement a symmetric baseline configuration with $w_k = 0.5$, $w_s = 0.5$, and $\tau_f = 0.5$.

Table 3: Assembly policy trade-off.

Assembly Mode	Avg Wait (ms)	Target Match Rate	Cold Start Ratio
Union	124.446	95.0%	5.0%
Intersection	393.523	100.0%	0.0%
Weighted	1308.378	55.0%	45.0%

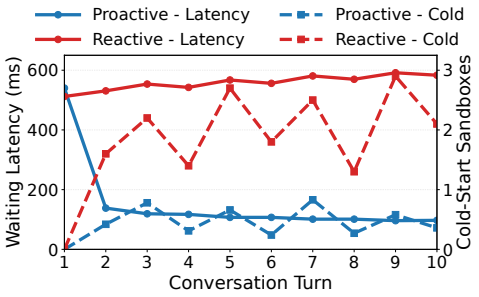


Figure 11: Dynamic execution profiling across a 10-turn conversation horizon, benchmarking per-turn average waiting latency (left) against the corresponding average cold-start sandbox activation count (right).

As quantified in Table 3, there is a clear trade-off between the aggressive *Union Assembly* and the conservative *Intersection* policy. *Union Assembly* prioritizes latency-masking agility, reducing the average waiting latency to just 124.45 ms at the cost of a slight 5.0% cold-start ratio. In contrast, the conservative *Intersection* policy eliminates cold starts (0.0%) by enforcing strict dual-router consensus, but at the cost of blocking the critical path and driving the average latency up to 393.52 ms (3.16× higher than Union).

The *Weighted* policy performs the worst, inflating latency to 1308.38 ms due to a 45.0% cold-start ratio. This failure stems from the structural misalignment between keyword tokens and high-dimensional semantics. Without dynamic re-normalization, shifting context distributions under multi-turn reasoning cause the combined scores to drift, frequently failing to clear the static $\tau_f = 0.5$ threshold. These results validate our choice of a *Union-first* assembly design to maximize latency-masking performance while maintaining practical routing correctness.

5.3.2 Effectiveness of Stochastic Sandbox Prefetching. To isolate the contribution of stochastic prefetching under multi-turn agent execution, we evaluate Laplace with two deployment variants under an identical planning workload:

- **Laplace-Reactive:** A routing-only baseline that performs inline token-level intent detection but does not use cross-step transition prediction.
- **Laplace-Proactive:** The full design that couples online hybrid routing with a stochastic Markovian prefetcher over the sandbox dependency graph (SDG), enabling asynchronous sandbox preparation before the next step.

Fig. 11 reports per-turn average waiting latency and cold-start counts over a 10-turn horizon. During Turn 1, both variants show

Table 4: Semantic cache trade-off.

Setting	Avg Wait (ms)	Hit Rate	Bypass Ratio
No Cache	412.78	0.0%	0.0%
Exact-Match Cache	233.64	33.6%	100.0%
Semantic Cache ($\tau = 0.6$)	141.93	37.4%	84.8%

similar latency (512.06 ms vs. 540.06 ms), since no historical transition signal is available to initialize the Markov predictor. From Turn 2 onward, the two trajectories diverge sharply. *Laplace-Proactive* reduces average waiting latency from 540.06 ms to 138.14 ms at Turn 2 and further to 97.14 ms by Turn 10, while *Laplace-Reactive* increases to 583.26 ms at Turn 10. This corresponds to a 6.0× end-horizon latency reduction. The mechanism is consistent with the cold-start telemetry. Under *Laplace-Reactive*, average cold-start count peaks at 2.90 instances per turn (Turn 9), indicating repeated exposure to sandbox initialization cost on the critical path. In contrast, *Laplace-Proactive* keeps per-turn cold starts within 0.24–0.83, effectively masking startup latency through early scheduling. These results show that Laplace improves multi-turn responsiveness through accurate, low-overhead temporal prefetching rather than infrastructure over-provisioning.

5.3.3 Effectiveness of Semantic Cache. To quantify the impact of our semantic cache, we benchmark three variants under the same repeated-request workload over 100 runs:

- **No Cache:** that executes every repeated request from scratch.
- **Exact-Match Cache:** A strict cache that matches on tool identity and normalized argument equality.
- **Semantic Cache:** reusing results when the tool identity matches and semantic similarity exceeds the threshold.

As shown in Table 4, while exact-match caching improves performance over *No Cache*, semantic caching captures a wider envelope of near-duplicate requests by tolerating surface-form variations. Compared to *No Cache*, semantic caching achieves a 2.91× speedup in average waiting latency (dropping from 412.78 ms to 141.93 ms) and increases the cache hit rate to 37.4% .

However, sematic caching is slightly lower than exact matching because a small fraction of semantic hits are not sufficiently reliable to skip sandbox setup and must fall back to normal execution after validation. This behavior is consistent with the intended design: semantic equivalence expands the reusable request space, but some loose matches still require conservative verification before execution can be bypassed.

5.3.4 Effectiveness of Out-of-Band Data Transmission. To evaluate the architectural efficiency of our control-and-data plane separation, we isolate the data transmission overhead by benchmarking two distinct transport paradigms across an exponential data payload spectrum scaling from 1.00 MB to 1000.00 MB:

- **Out-of-Band:** Our proposed out-of-band transport mechanism. It completely bypasses the control-plane RPC tunnel by writing bulky state payloads directly to a dedicated local shared-memory substrate or zero-copy virtualized host-guest ring buffers, passing only lightweight, fixed-size references over the wire.
- **JSON-RPC:** The standard baseline paradigm utilized in conventional agent runtimes. It marshals multi-modal data payloads directly into the inline runtime execution stream, forcing the control-plane to serialize and transport raw data matrices synchronously via text-based JSON-RPC network primitives.

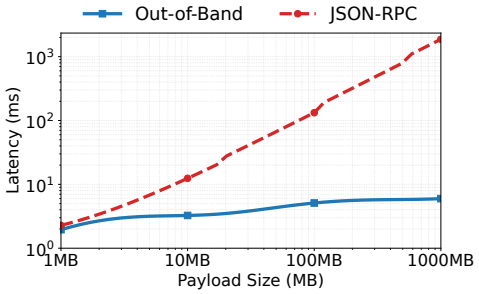


Figure 12: Data transmission latency scaling profiles under varying payload sizes, comparing Laplace's out-of-band data transmission against standard JSON-RPC serialization.

As shown in Fig. 12, the empirical measurements reveal a clear scaling divergence between the two protocols across an exponential payload spectrum. At the baseline threshold (1.00 MB), both configurations demonstrate sub-3 millisecond performance, with *Out-of-Band* maintaining a slight edge (1.95 ms vs. 2.28 ms). However, as the payload expands, the *JSON-RPC* pipeline suffers a catastrophic linear performance degeneration; throttled by heavy serialization bottlenecks, its latency grows to 132.45 ms at 100.00 MB and reaches 1873.16 ms at the 1000.00 MB boundary. Conversely, *Out-of-Band* demonstrates a near-constant $O(1)$ scaling profile, drifting to only 5.97 ms at the 1 GB boundary—a 313.55× latency reduction.

This performance gap stems from eliminating critical-path serialization and memory copying. In multi-turn workflows, agents frequently exchange large multi-modal states like high-dimensional vectors or media files. Standard JSON-RPC stalls the control loop due to synchronous serialization, whereas Laplace's out-of-band design decouples control signals from raw payload routing, reducing data transfer to a constant-time reference-passing operation. These results demonstrate that separating the control and data planes keeps Laplace's orchestration overhead minimal and independent of payload size.

6 DISCUSSION

Architectural Overhead. A critical concern in predictive serving runtimes is whether the system-level orchestration mechanisms introduce non-negligible processing penalties onto the critical path. In Laplace, this control-plane overhead is thoroughly isolated from the GPU-bound inference loop through strict out-of-band execution and parallel routing mechanics. While the token-level scanning runs with deterministic $O(1)$ complexity, the inherently heavier semantic retrieval engine is offloaded to dedicated background CPU worker threads. Because the intent router utilizes an *Union Assembly* ($\cup$), the critical path never blocks for late-arriving semantic embeddings; any early fast-path trigger instantly dispatches the activation primitive within microseconds. Combined with the prefetching daemon–which evaluates low-dimension Markov matrix transitions bounded by the small cardinality of active sandboxes–Laplace restricts its control-plane telemetry to tens of microseconds, securing a nearly zero-cost latency impact on token generation.

Ecosystem Generalization. We position Laplace as a runtime middleware layer operating between upstream reasoning agents and downstream execution environments. This design leverages two widely available capabilities in modern agent ecosystems [9, 15, 43]: streaming token-level outputs from upstream frameworks, and standardized tool interfaces provided by protocols such as MCP [5]. As a result, Laplace requires no additional system-level constraints beyond what is already supported in existing agent and sandbox orchestration stacks.

Importantly, this execution abstraction extends beyond inference-time serving to agentic reinforcement learning (Agentic RL) frameworks, such as VERL [33] and Slime [49], where rollout generation and environment interaction follow a structurally similar loop. In these Agentic RL settings, Laplace 's runtime optimizations—namely *intent-aware sandbox prewarming* and *stochastic sandbox prefetching*—can be seamlessly adapted to mask environment initialization and interaction overheads during training rollouts with modest integration effort.

Limitations and Future Work. Despite its performance gains, Laplace exhibits certain architectural boundaries. On the control plane, our prefetcher assumes first-order history dependence via a Markov Chain model, which may suffer from diminished prediction accuracy during open-ended, long-horizon agent workflows. Crucially, Laplace inherently mitigates this via its hierarchical, two-tiered preparation: any inter-step prefetch miss gracefully falls back to the intra-step intent-aware prewarming during real-time token streaming. This design guarantees that worst-case environment latency remains tightly bounded within the LLM's decoding phase. Future work will investigate online Graph Neural Networks (GNNs) to adaptively capture non-linear transition patterns.

On the data plane, our `mmap`-based transmission requires co-locating the Agent Engine and sandboxes within the same host boundary. While this design seamlessly aligns with mainstream multi-tenant serving topologies (e.g., Kubernetes Pod IPC sharing or Sidecar patterns) and leverages robust container-level namespace and cgroup security isolation, it restricts single-session cross-node scaling. To support large-scale distributed clusters, we plan to integrate RDMA-assisted zero-copy transmission, extending our out-of-band data plane into disaggregated cloud infrastructures.

7 RELATED WORK

Lightweight Sandbox Runtimes. The systems community has actively explored lightweight isolation mechanisms to mitigate the physical instantiation cost of serverless execution environments. Notable advancements include micro-virtual machines (MicroVMs) like Firecracker [1] and RunD [19], WebAssembly (Wasm) [41] runtimes, and process-level snapshotting/forking frameworks such as FaaSnap [6] and TrEnv-X [12]. These infrastructure-level systems focus on minimizing localized host setup or exploiting hardware-assisted remote memory pools (e.g., CXL/RDMA) to share and reuse physical sandboxes across tenants.

Crucially, these low-layer container optimizations are entirely orthogonal to Laplace. While microVM snapshots or repurposable sandboxes successfully compress the physical infrastructure boot time to milliseconds, application-layer handshakes (such as MCP discovery hooks) and environment attachment bottlenecks still natively linger on the critical runtime path. Laplace operates at a higher, application-perceptive orchestration layer; it complements these physical speedups by exploiting a distinct temporal dimension—overlapping control-plane scheduling with streaming token generation—to fully mask, rather than compress, the intrinsic readiness latency.

Serverless Predictive Prewarming. Predictive prewarming is a widely embraced technique in standard serverless frameworks to eradicate the notorious tail-latency penalties of tenant cold starts. State-of-the-art prefetching daemons (e.g., Mitosis [42] and IceBreaker [30]) primarily depend on historical time-series analytics, statistical invocation frequency histograms, or temporal correlation clustering to predictively prepare idle container runtimes.

However, these classical prewarming paradigms inherently assume that incoming requests follow independent and identically distributed arrival models or deterministic time-triggered patterns. This core assumption breaks down completely under emerging LLM Agent workloads. In multi-turn autonomous agent reasoning loops, tool invocations are dynamically, autonomously, and non-linearly determined by the LLM's fluid context trajectory, presenting complex execution dependency horizons. Laplace bridges this gap by co-designing the serving infrastructure with explicit agent behavioral characteristics, introducing a stochastic Markovian predictive framework built over a sandbox dependency graph (SDG) to model real-time autonomous state transitions.

LLM Serving and Agent Orchestration. Accelerating the end-to-end execution of Large Language Models has driven extensive research across the AI systems spectrum. On the model-serving boundary, mainstream runtimes such as vLLM [14] and SGLang [48] optimize GPU-internal kernel execution, memory caching via PagedAttention, and automated speculative decoding. Parallel to this, application-level agent orchestration platforms like AgentScope [9], AutoGen [43] and LangGraph [15] provide modular abstractions for programming multi-turn autonomous multi-agent teams.

Nevertheless, existing LLM serving engines fundamentally treat model execution as a self-contained GPU computing unit, entirely oblivious to the physical host-plane environment friction when interacting with external tools. Conversely, high-level agent frameworks lack system-level visibility into cloud-native host topologies, resulting in uncoordinated data transfer and execution stalls. Laplace operates as a runtime middleware layer that bridges this gap, enabling co-optimized out-of-band control-plane predictive routing and shared-memory data transmission across heterogeneous multi-tenant host execution boundaries.

8 CONCLUSIONS

We present Laplace, a predictive execution runtime for LLM-based agent systems that reduces tail latency and resource inefficiency in multi-tenant environments. By analyzing the end-to-end agent execution step, we find that inefficiencies primarily arise from rigid dependencies among reasoning, environment initialization, and sandbox execution. Laplace addresses this through a unified design that enables temporal overlap between LLM execution and sandbox setup, anticipates future sandbox needs across steps to reduce cross-step cold-start overheads, and eliminates redundant computation and communication through reuse-aware execution

and out-of-band data transport. Laplace is implemented on top of AgentScope [9] and evaluated it under highly concurrent multi-turn workloads. Results show up to a 2.9× reduction in P99 latency and 45.9% lower peak memory usage, with a 97.9% prewarming hit rate. These results demonstrate that exploiting temporal overlap across the agent execution loop is an effective and general mechanism for improving efficiency in large-scale agent deployments.

ACKNOWLEDGMENT

We would very much like to thank our shepherd, Prof. Purushottam Kulkarni, and the anonymous SoCC reviewers for their valuable and constructive comments. Special thanks must go to the overall AI platform team at Kuaishou Inc. and the members in RAIDS Lab at Beihang University, for their constant support, collaborative contribution and countless technical discussion.

This work is supported in part by National Key R&D Program of China (Grant No. XXXXXXXXXXXXXXX), in part by the National Natural Science Foundation of China (Grant XXXXXXXX, No. XXXXXXXXX), in part by the Beijing Natural Science Foundation (Grant No. XXXXXXX). For any correspondence, please refer to the project lead and coordinator Dr. Renyu Yang (renyuyang@buaa.edu.cn).

REFERENCES

[1] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX symposium on networked systems design and implementation (NSDI 20)*. 419–434.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.

[3] Alibaba Cloud. 2026. *DashScope - Alibaba Cloud: AI and Cloud Computing Services.* https://dashscope.console.aliyun.com/ Retrieved June, 2026.

[4] Amazon Web Services. 2026. *Amazon Bedrock AgentCore- AWS.* https://aws.amazon.com/bedrock/agentcore/ Retrieved June, 2026.

[5] Anthropic PBC. 2024. *Introducing the Model Context Protocol.* https://www.anthropic.com/news/model-context-protocol Retrieved June, 2026.

[6] Lixiang Ao, George Porter, and Geoffrey M Voelker. 2022. Faasnap: Faas made fast using snapshot-based vms. In *Proceedings of the Seventeenth European Conference on Computer Systems*. 730–746.

[7] Datalayer. 2026. *Jupyter MCP Server.* https://github.com/datalayer/jupyter-mcp-server Retrieved June, 2026.

[8] Dong Du, Tianyi Yu, Yubin Xia, Binyu Zang, Guanglu Yan, Chenggang Qin, Qixuan Wu, and Haibo Chen. 2020. Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 467–481.

[9] Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, et al. 2024. Agentscope: A flexible yet robust multi-agent platform. *arXiv preprint arXiv:2402.14034* (2024).

[10] GitHub. 2026. *MCP Registry.* https://github.com/mcp Retrieved June, 2026.

[11] Google Cloud. 2026. *Gemini Enterprise Agent Platform.* https://docs.cloud.google.com/gemini-enterprise-agent-platform/scale/sandbox Retrieved June, 2026.

[12] Jialiang Huang, Teng Ma, Zheng Liu, Sixing Lin, Kang Chen, Jinlei Jiang, Xia Liao, Yingdi Shan, Yongwei Wu, Ning Zhang, Mengting Lu, Tao Ma, Haifeng Gong, and Mingxing Zhang. 2026. TrEnv-X: Transparently Share Serverless Execution Environments Across Different Functions and Nodes. *ACM Trans. Comput. Syst.* (March 2026). https://doi.org/10.1145/3805475 Just Accepted.

[13] Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. 2017. Bag of Tricks for Efficient Text Classification. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*. Association for Computational Linguistics, 427–431.

[14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.

[15] Langchain-ai. 2026. *LangGraph: Agent Orchestration Framework for Reliable AI Agents.* https://www.langchain.com/langgraph Retrieved June, 2026.

[16] Pierre-Simon Laplace. 1812. *Théorie analytique des probabilités.* Courcier, Paris. http://gallica.bnf.fr/ark:/12148/bpt6k88764q

[17] Tianyu Li, Badrish Chandramouli, Philip A Bernstein, and Samuel Madden. 2024. Distributed Speculative Execution for Resilient Cloud Applications. *arXiv preprint arXiv:2412.13314* (2024).

[18] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. 2024. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth* 1, 1 (2024), 9.

[19] Zijun Li, Jiagan Cheng, Quan Chen, Eryu Guan, Zizheng Bian, Yi Tao, Bin Zha, Qiang Wang, Weidong Han, and Minyi Guo. 2022. {RunD}: A lightweight secure container runtime for high-density deployment and high-concurrency startup in serverless computing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 53–68.

[20] Zijun Li, Chuhao Xu, Quan Chen, Jieru Zhao, Chen Chen, and Minyi Guo. 2023. Dataflower: Exploiting the data-flow paradigm for serverless workflow orchestration. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*. 57–72.

[21] David H Liu, Amit Levy, Shadi Noghabi, and Sebastian Burckhardt. 2023. Doing more with less: Orchestrating serverless applications without an orchestrator. In *20th USENIX symposium on networked systems design and implementation (NSDI 23)*. 1505–1519.

[22] Fangming Lu, Xingda Wei, Zhuobin Huang, Rong Chen, Minyu Wu, and Haibo Chen. 2024. Serialization/deserialization-free state transfer in serverless workflows. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 132–147.

[23] Ashraf Mahgoub, Edgardo Barsallo Yi, Karthick Shankar, Sameh Elnikety, Somali Chaterji, and Saurabh Bagchi. 2022. {ORION} and the three rights: Sizing, bundling, and prewarming for serverless {DAGs}. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 303–320.

[24] Kai Mei, Xi Zhu, Wujiang Xu, Mingyu Jin, Wenyue Hua, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. [n. d.]. AIOS: LLM Agent Operating System. In *Second Conference on Language Modeling*.

[25] Microsoft. 2026. *Azure Container Apps Sandboxes.* https://sandboxes.azure.com/ Retrieved June, 2026.

[26] Microsoft. 2026. *Playwright MCP.* https://github.com/microsoft/playwright#playwright-mcp Retrieved June, 2026.

[27] Neo4j. 2026. *Neo4j official MCP Server.* https://github.com/neo4j/mcp Retrieved June, 2026.

[28] OpenAGS. 2026. *openags/paper-search-mcp: MCP, CLI, Skills for searching and downloading academic papers from multiple sources like arXiv, PubMed, bioRxiv, etc.* https://github.com/openags/paper-search-mcp Retrieved June, 2026.

[29] Carlo Puliafito, Claudio Cicconetti, Marco Conti, Enzo Mingozzi, and Andrea Passarella. 2022. Stateless or stateful FaaS? I'll take both!. In *2022 IEEE International Conference on Smart Computing (SMARTCOMP)*. IEEE, 62–69.

[30] Rohan Basu Roy, Tirthak Patel, and Devesh Tiwari. 2022. Icebreaker: Warming serverless functions better with heterogeneity. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 753–767.

[31] sentence-transformers. 2026. *all-MiniLM-L6-v2.* https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 Retrieved June, 2026.

[32] Mohammad Shahrad, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX annual technical conference (USENIX ATC 20)*. 205–218.

[33] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2024. HybridFlow: A Flexible and Efficient RLHF Framework. *arXiv preprint arXiv: 2409.19256* (2024).

[34] Karen Sparck Jones. 1972. A statistical interpretation of term specificity and its application in retrieval. *Journal of documentation* 28, 1 (1972), 11–21.

[35] Jovan Stojkovic, Tianyin Xu, Hubertus Franke, and Josep Torrellas. 2023. Mxfaas: Resource sharing in serverless environments for parallelism and efficiency. In *Proceedings of the 50th annual international symposium on computer architecture*. 1–15.

[36] Jovan Stojkovic, Tianyin Xu, Hubertus Franke, and Josep Torrellas. 2023. Specfaas: Accelerating serverless applications with speculative function execution. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 814–827.

[37] Yifan Sui, Hanfei Yu, Yitao Hu, Jianxun Li, and Hao Wang. 2024. Pre-warming is not enough: Accelerating serverless inference with opportunistic pre-loading. In *Proceedings of the 2024 ACM Symposium on Cloud Computing*. 178–195.

[38] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2025. Towards end-to-end optimization of llm-based applications with ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1302–1316.

[39] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6

(2024), 186345.

[40] Zhenting Wang, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, Alireza Rezazadeh, Ankit Shah, Yujia Bao, et al. 2025. Mcp-bench: Benchmarking tool-using llm agents with complex real-world tasks via mcp servers. *arXiv preprint arXiv:2508.20453* (2025).

[41] WebAssembly. 2026. *WebAssembly*. https://webassembly.org/ Retrieved June, 2026.

[42] Xingda Wei, Fangming Lu, Tianxia Wang, Jinyu Gu, Yuhan Yang, Rong Chen, and Haibo Chen. 2023. No provisioned concurrency: Fast {RDMA-codesigned} remote fork for serverless computing. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 497–517.

[43] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155* (2023).

[44] Chuhao Xu, Yiyu Liu, Zijun Li, Quan Chen, Han Zhao, Deze Zeng, Qian Peng, Xueqi Wu, Haifeng Zhao, Senbo Fu, et al. 2024. Faasmem: Improving memory efficiency of serverless computing with memory pool architecture. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 331–348.

[45] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).

[46] Hanfei Yu, Rohan Basu Roy, Christian Fontenot, Devesh Tiwari, Jian Li, Hong Zhang, Hao Wang, and Seung-Jong Park. 2024. Rainbowcake: Mitigating cold-starts in serverless with layer-wise container caching and sharing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 335–350.

[47] Yihui Zhang, Han Shen, Renyu Yang, Di Tian, Yuxi Luo, Menghao Zhang, Li Li, Chunming Hu, Tianyu Wo, Chengru Song, et al. 2025. Cauchy: A Cost-Efficient LLM Serving System through Adaptive Heterogeneous Deployment. In *Proceedings of the 2025 ACM Symposium on Cloud Computing*. 881–893.

[48] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.

[49] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. 2025. slime: An LLM post-training framework for RL Scaling. https://github.com/THUDM/slime. GitHub repository. Corresponding author: Xin Lv.